

SSM框架

1. Spring框架中的单例bean是线程安全的吗？

候选人：

不是线程安全的。当多用户同时请求一个服务时，容器会给每个请求分配一个线程，这些线程会并发执行业务逻辑。如果处理逻辑中包含对单例状态的修改，比如修改单例的成员属性，就必须考虑线程同步问题。Spring框架本身并不对单例bean进行线程安全封装，线程安全和并发问题需要开发者自行处理。

通常在项目中使用的Spring bean是不可变状态（如Service类和DAO类），因此在某种程度上可以说Spring的单例bean是线程安全的。如果bean有多种状态（如ViewModel对象），就需要自行保证线程安全。最简单的解决办法是将单例bean的作用域由“singleton”变更为“prototype”。

2. 什么是AOP？

候选人：

AOP，即面向切面编程，在Spring中用于将那些与业务无关但对多个对象产生影响的公共行为和逻辑抽取出来，实现公共模块复用，降低耦合。常见的应用场景包括公共日志保存和事务处理。

3. 你们项目中有没有使用到AOP？

候选人：

我们之前在后台管理系统中使用AOP来记录系统操作日志。主要思路是使用AOP的环绕通知和切点表达式，找到需要记录日志的方法，然后通过环绕通知的参数获取请求方法的参数，例如类信息、方法信息、注解、请求方式等，并将这些参数保存到数据库。

4. Spring中的事务是如何实现的？

候选人：

Spring实现事务的本质是利用AOP完成的。它对方法前后进行拦截，在执行方法前开启事务，在执行完目标方法后根据执行情况提交或回滚事务。

5. Spring中事务失效的场景有哪些？

候选人：

在项目中，我遇到过几种导致事务失效的场景：

1. 如果方法内部捕获并处理了异常，没有将异常抛出，会导致事务失效。因此，处理异常后应该确保异常能够被抛出。
2. 如果方法抛出检查型异常（checked exception），并且没有在 `@Transactional` 注解上配置 `rollbackFor` 属性为 `Exception`，那么异常发生时事务可能不会回滚。
3. 如果事务注解的方法不是公开（public）修饰的，也可能导致事务失效。

6. Spring的bean的生命周期？

候选人：

Spring中bean的生命周期包括以下步骤：

1. 通过 `BeanDefinition` 获取bean的定义信息。
2. 调用构造函数实例化bean。
3. 进行bean的依赖注入，例如通过setter方法或 `@Autowired` 注解。
4. 处理实现了 `Aware` 接口的bean。
5. 执行 `BeanPostProcessor` 的前置处理器。
6. 调用初始化方法，如实现了 `InitializingBean` 接口或自定义的 `init-method`。
7. 执行 `BeanPostProcessor` 的后置处理器，可能在这里产生代理对象。
8. 最后是销毁bean。

7. Spring中的循环引用？

候选人：

循环依赖发生在两个或两个以上的bean互相持有对方，形成闭环。Spring框架允许循环依赖存在，并通过三级缓存解决大部分循环依赖问题：

1. 一级缓存：单例池，缓存已完成初始化的bean对象。
2. 二级缓存：缓存尚未完成生命周期的早期bean对象。
3. 三级缓存：缓存 `ObjectFactory`，用于创建bean对象。

8. 那具体解决流程清楚吗？

候选人：

解决循环依赖的流程如下：

1. 实例化A对象，并创建 `ObjectFactory` 存入三级缓存。

2. A在初始化时需要B对象，开始B的创建逻辑。
3. B实例化完成，也创建 `ObjectFactory` 存入三级缓存。
4. B需要注入A，通过三级缓存获取 `ObjectFactory` 生成A对象，存入二级缓存。
5. B通过二级缓存获得A对象后，B创建成功，存入一级缓存。
6. A对象初始化时，由于B已创建完成，可以直接注入B，A创建成功存入一级缓存。
7. 清除二级缓存中的临时对象A。

9. 构造方法出现了循环依赖怎么解决？

候选人：

由于构造函数是bean生命周期中最先执行的，Spring框架无法解决构造方法的循环依赖问题。可以使用 `@Lazy` 懒加载注解，延迟bean的创建直到实际需要时。

10. SpringMVC的执行流程？

候选人：

SpringMVC的执行流程包括以下步骤：

1. 用户发送请求到前端控制器 `DispatcherServlet`。
2. `DispatcherServlet` 调用 `HandlerMapping` 找到具体处理器。
3. `HandlerMapping` 返回处理器对象及拦截器（如果有）给 `DispatcherServlet`。
4. `DispatcherServlet` 调用 `HandlerAdapter`。
5. `HandlerAdapter` 适配并调用具体处理器（Controller）。
6. Controller执行并返回 `ModelAndView` 对象。
7. `HandlerAdapter` 将 `ModelAndView` 返回给 `DispatcherServlet`。
8. `DispatcherServlet` 传给 `ViewResolver` 进行视图解析。
9. `ViewResolver` 返回具体视图给 `DispatcherServlet`。
10. `DispatcherServlet` 渲染视图并响应用户。

11. Springboot自动配置原理？

候选人：

Spring Boot的自动配置原理基于 `@SpringBootApplication` 注解，它封装了 `@SpringBootConfiguration`、`@EnableAutoConfiguration` 和 `@ComponentScan`。
`@EnableAutoConfiguration` 是核心，它通过 `@Import` 导入配置选择器，读取 `META-`

INF/spring.factories 文件中的类名，根据条件注解决定是否将配置类中的Bean导入到Spring容器中。

12. Spring 的常见注解有哪些？

候选人：

Spring的常见注解包括：

1. 声明Bean的注解：@Component、@Service、@Repository、@Controller。
2. 依赖注入相关注解：@Autowired、@Qualifier、@Resource。
3. 设置作用域的注解：@Scope。
4. 配置相关注解：@Configuration、@ComponentScan、@Bean。
5. AOP相关注解：@Aspect、@Before、@After、@Around、@Pointcut。

13. SpringMVC常见的注解有哪些？

候选人：

SpringMVC的常见注解有：

- @RequestMapping：映射请求路径。
- @RequestBody：接收HTTP请求的JSON数据。
- @RequestParam：指定请求参数名称。
- @PathVariable：从请求路径中获取参数。
- @ResponseBody：将Controller方法返回的对象转化为JSON。
- @RequestHeader：获取请求头数据。
- @PostMapping、@GetMapping 等。

14. Springboot常见注解有哪些？

候选人：

Spring Boot的常见注解包括：

- @SpringBootApplication：由 @SpringBootConfiguration、@EnableAutoConfiguration 和 @ComponentScan 组成。
- 其他注解如 @RestController、@GetMapping、@PostMapping 等，用于简化Spring MVC的配置。

15. MyBatis执行流程？

候选人：

MyBatis的执行流程如下：

1. 读取MyBatis配置文件 `mybatis-config.xml`。
2. 构造会话工厂 `SqlSessionFactory`。
3. 会话工厂创建 `SqlSession` 对象。
4. 操作数据库的接口， `Executor` 执行器。
5. `Executor` 执行方法中的 `MappedStatement` 参数。
6. 输入参数映射。
7. 输出结果映射。

16. Mybatis是否支持延迟加载？

候选人：

MyBatis支持延迟加载，即需要用到数据时才加载。可以通过配置文件中的 `lazyLoadingEnabled` 配置启用或禁用延迟加载。

17. 延迟加载的底层原理知道吗？

候选人：

延迟加载的底层原理主要使用CGLIB动态代理实现：

1. 使用CGLIB创建目标对象的代理对象。
2. 调用目标方法时，如果发现是null值，则执行SQL查询。
3. 获取数据后，设置属性值并继续查询目标方法。

18. Mybatis的一级、二级缓存用过吗？

候选人：

MyBatis的一级缓存是基于 `PerpetualCache` 的HashMap本地缓存，作用域为Session，默认开启。二级缓存需要单独开启，作用域为Namespace或mapper，默认也是采用 `PerpetualCache`，HashMap存储。

19. Mybatis的二级缓存什么时候会清理缓存中的数据？

候选人：

当作用域（一级缓存Session/二级缓存Namespaces）进行了新增、修改、删除操作后，默认该作用域下所有select中的缓存将被清空。